

Previous System Examples

Example 1: [Incorporating External Knowledge through Pre-training for Natural Language to Code Generation](#)

This first system is detailed in a paper published in April of 2020. Knowing that developers who run into problems typically go to the internet for answers, the goal of the project was to explore the effect of taking data from multiple online sources and using it as the knowledge base for a model built for NL-to-code generation.

For their sources of data, the team used data from StackOverflow and documentation for some of the more common software libraries, such as the Python standard library. All the data used was in the form of natural language/code pairs. They started by pre-training a model on these automatically retrieved data, then they fine-tuned it on a similar smaller dataset that was manually built and human-annotated to focus on specific test cases.

This project is similar to the first steps in our project – gathering data. We had previously discussed scraping documentation from StackOverflow and Python/Pandas libraries, so knowing that this technique has been used for a similar use case gives us a promising start.

Example 2: [PandasAI](#)

PandasAI is another system that was built to accomplish a task similar to what we intend to do. It is an open-source system designed to make data analysis easier for users by allowing them to ask natural-language questions about a pandas DataFrame and receive computed results. Its goal is to remove the need for users to write code themselves, instead translating natural language queries automatically into executable Python code.

PandasAI uses large language models to generate code based on user prompts. It incorporates DataFrame schema info, such as column names and data types, into structured prompts before sending them to the model. The code that is generated is then

executed in a controlled Python environment, and the results (which can include tables and plots) are returned to the user.

This system is similar to our proposed project because it also performs natural language to code translation specifically for data analysis tasks. This system is especially useful to us because it includes the execution of the generated code rather than just returning the code created by the model.

Example 3: [TRANX](#)

TRANX is a system designed to generate code from natural language by constructing abstract syntax trees (ASTs). Its goal was to improve the syntactic correctness and generalizability of natural language to code generation models by using structured representation of programs instead of treating the code as plain text.

This system uses a transition-based neural parser to map English descriptions into ASTs, which are then converted into executable code. It was trained on code generation and semantic parsing benchmarks, such as Python and SQL tasks, using pairs of natural language prompts and correct code.

This system is similar to what we intend to do because it also was created to translate natural language into executable code. Although the structure of the systems differ they have similar ultimate goals.

Techniques Used

Documentation scraping is used to gather data which relates natural language problems or use cases to specific lines of code. This is useful for our project because it is how our model will learn what functions to use and how to use them to solve problems given as natural language.

Available Tools and Frameworks

- For scraping and dataset creation, we would use [Beautiful Soup](#) and the [Stack Exchange API](#) for structured data extraction.
- For the core language understanding and code generation component, we would use the [Hugging Face Transformers](#) library for loading and fine-tuning an

instruction-tuned code model (e.g., Code LLaMA or similar) and the datasets module to preprocess our Stack Overflow and documentation-derived QA pairs into a uniform (natural language question, Python code) format.

- For training and evaluation workflows, we would use PyTorch's torch.nn for model fine-tuning and torch.utils.data for batching and dataset pipelines.
- For NLP preprocessing (cleaning scraped text, tokenization, etc.), we would use NLTK.
- Code execution would occur in a sandboxed Python runtime using the Subprocess library. This executed code will rely on common data science libraries like Pandas and Matplotlib.

Summary of our System

Our system will function as a natural-language-driven data analysis agent that translates user questions about uploaded datasets into executable Python code and returns interpretable results (numerical summaries, tables, or visualizations). The task it accomplishes is semantic mapping from plain English data analysis queries (e.g., “What is the average sales per region?”) to correct, executable Python code using libraries such as Pandas and Matplotlib. The main difference between that which has been done many times before and our own system is that the output shown to the user is not raw code, but the computed analytical result and a natural language explanation of what was done.

The system will combine instruction-tuned large language models. Training data will consist of cleaned documentation examples and Stack Overflow QA pairs converted into standardized (natural language question, Python code) format. When a user uploads a file (CSV or JSON), the backend loads it into a Pandas DataFrame, constructs a structured prompt containing schema information (column names, types, preview rows), and feeds this along with the user’s query into the model. The generated Python code is executed in a sandboxed local environment, and the resulting objects (scalars, DataFrames, or plots) are shown to the user.